

Programação Funcional

Prof. Malbarbo

6 de agosto de 2026

Conteúdo

1	Por que usar o paradigma funcional?	1
2	Fundamentos da Programação Funcional	3

Dentro da programação não temos apenas uma forma de implementar um programa, temos alguns diferentes paradigmas que podemos usar:

- Programação Imperativa – Os programas são descritos com sentenças que modificam o estado do programa
- Programação Orientada a Objetos – Os programas são descritos organizados em torno de objetos, classes e interações
- Programação Funcional – Os programas são descritos com aplicação e composição de funções
 - Evita mudança de estado (mudança do valor das variáveis)
 - Evita efeitos colaterais, que são quaisquer efeitos que sejam observáveis além do valor de saída da função, como a mudança dos parâmetros e variáveis globais, exceções, entrada e saída, etc.

1 Por que usar o paradigma funcional?

Um paradigma de programação é uma ferramenta, e conhecer várias ferramentas permite utilizar a mais adequada para cada problema, e muitas vezes o Python e a programação imperativa têm problemas.

- Mudança de estados:
 - Devido a forma com que uma lista é alocada em Python, inconsistências acontecem, como quando alocamos:

```
>>> lst = [0] * 3
>>> lst
[0, 0, 0]
```

```
>>> lst[1] = 10
>>> lst
[0, 10, 0]
```

– Porém, ao alocar uma lista dentro de lista:

```
>>> lst = [[]] * 3
>>> lst
[[], [], []]
>>> lst[1].append(2)
>>> lst
[[2], [2], [2]]
```

– Além disso, quando modificando uma lista usando `adiciona_todos`

```
def adiciona_todos(dest: list[int], fonte: list[int]):
    '''
    Adiciona todos os elementos de *fonte* no final de
    *dest*
    '''
    for x in fonte:
        dest.append(x)
```

* Ao modificar `lst` usando uma lista diferente não temos problemas

```
>>> lst = [4, 3, 1]
>>> adiciona_todos(lst, [6, 2])
>>> lst
[4, 3, 1, 6, 2]
```

* Porém, ao adicionar `lst` nela mesma:

```
>>> adiciona_todos(lst, lst)
>>> lst
```

A execução não acaba, visto que a lista se automodifica infinitamente.

- Efeitos colaterais

– Dentro de uma função, como vimos, é possível mudar o valor de um dos parâmetros de entrada, dependendo da alteração, o mesmo código, dependendo da ordem que são realizadas as funções, temos resultados diferentes.

```
def soma_indices(lst: list[int], a: int, b: int) -> int:
    return indice(lst, b) + indice(lst, a)

def soma_indices(lst: list[int], a: int, b: int) -> int:
    return indice(lst, a) + indice(lst, b)
```

Não é possível que as duas definições são equivalentes sem olhar o código da função `indice`. Se ela tiver efeitos colaterais, então as definições não podem ser equivalentes.

- Com os efeitos colaterais dificulta pensar localmente sobre o funcionamento do código, sem eles, podemos fazer isso.

2 Fundamentos da Programação Funcional

O paradigma de programação funcional é baseado na definição e aplicação de funções

- Função: Conjunto de expressões que mapeia valores de entrada para valores de saída
- Expressão: Entidade sintática que quando avaliada (reduzida) produz um valor
- Sentença: Operação que não retorna valor, apenas um efeito colateral (no Python, `if`, `for`, `while`, etc) – Não temos sentenças no paradigma funcional

Uma expressão consiste em:

- Um literal – Valor que é representado diretamente no código, em geral, os literais utilizados para criar valores de tipos primitivos.
- Uma função primitiva – Função suportada diretamente pela linguagem de programação

Tipos primitivos:

- No Gleam, os tipos primitivos sempre começam com letra maiúscula
- Numero inteiro (Int)
 - 1345
 - 9_876
- Numero com ponto flutuante (Float)
 - 2.65
 - 2.0e12
 - 7.4e-10
- Booleano (Bool)
 - True
 - False
- Strings (String)
 - "din uem"
 - "apenas \"um\" teste"

Funções primitivas:

- Operações com inteiros:

- + (`int.add`)

- - (`int.subtract`)

- * / % > >= <= == !=